

HW 2B: OWASP Juice Shop

Siva Sai Deepank Manoj

deeps45@tamu.edu

GitHub: <https://github.com/deeps45/juice-guard>

Part 1 — Secure Feature Design

I spent time on the live Juice Shop at preview.owasp-juice.shop, mostly around login and the Score Board. Three challenges stood out to me. **Login Admin** is an injection challenge where a crafted email can break a string-built SQL login query. To stop that, I would only use parameterized queries (or an ORM) and validate the email first so user input never becomes part of the query logic. **DOM XSS** shows how HTML/JavaScript in the page can run in someone else's browser. I would encode output, use `textContent` instead of `innerHTML`, and add a Content-Security-Policy. For **Forged Signed JWT** / weak auth, I would verify signatures with a strong secret, keep tokens short-lived, and rate-limit login attempts. Passwords should be stored with `bcrypt`, not plaintext.

Learnings (~100 words)

What surprised me most is how “normal” the Juice Shop login screen looks while still being easy to abuse if the backend concatenates SQL. Looking up Login Admin and DOM XSS on the Score Board made the risks concrete instead of abstract. I also realized authentication bugs are not only about passwords—token handling matters just as much. My takeaway for a secure registration/login design is simple: treat every field as hostile, keep queries parameterized, never render raw HTML from users, and hash passwords with `bcrypt` before they touch storage. Those few habits would have blocked most of what I saw in the challenges.

Password hashing example

```
const hash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(submittedPassword, user.passwordHash);
```

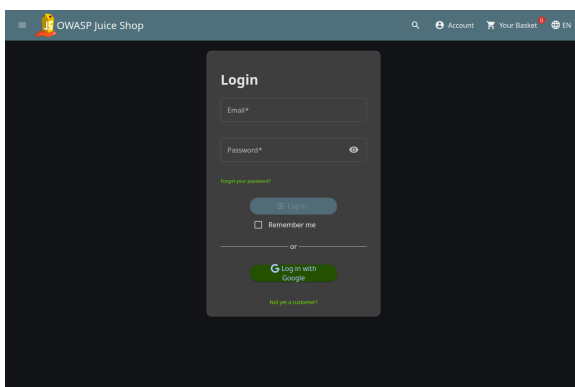


Figure 1. Juice Shop login page I inspected.

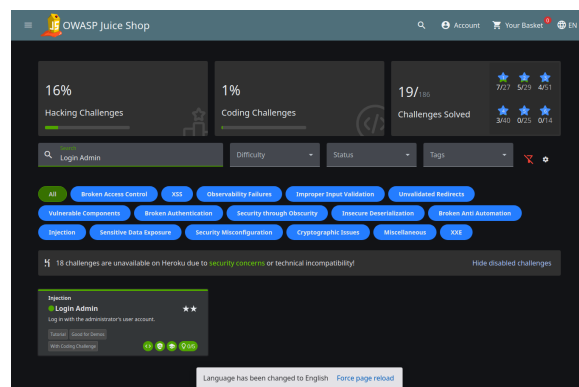


Figure 2. Score Board — Login Admin challenge.

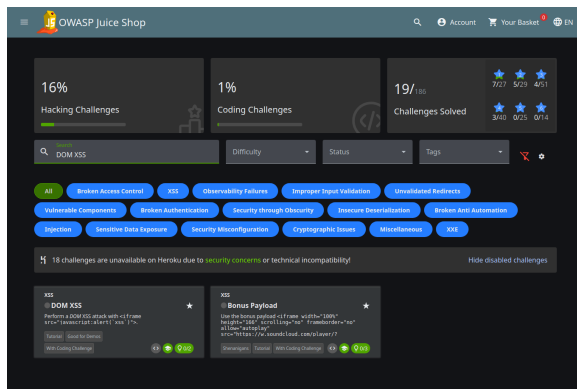


Figure 3. Score Board — DOM XSS challenge.

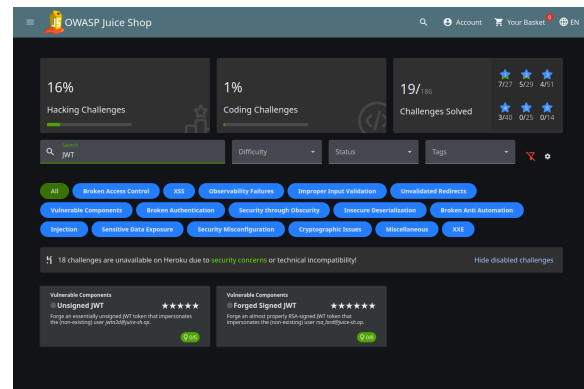


Figure 4. Score Board — JWT / broken-auth challenges.

Part 2 — Front-End Login Form

I built a simple login page in HTML, CSS, and JavaScript with a small Express server behind it. The form has email and password fields. On the client, app.js blocks empty submissions, checks that the email contains “@”, and requires the password to be at least eight characters. On the server, those checks run again more strictly, passwords are verified with bcrypt, and login also requires a CSRF token from /api/csrf. Status messages use textContent so reflected text cannot turn into HTML. I kept demo credentials in the README only, not on the page.

Learnings (~100 words)

Building the form made the Juice Shop lessons feel practical. Client-side checks are nice for users—they catch empty fields quickly—but they are easy to skip, so I treated the server as the real gate. Adding CSRF forced me to think about forged requests, not only bad passwords. Hashing with bcrypt was straightforward once I stopped thinking of “encryption” and started thinking of one-way hashing. Overall I learned that a “basic” login page still needs several layers: validation, safe rendering, password hashing, and request authenticity. The public repo and README document how to run everything locally.

Public GitHub repository: <https://github.com/deeps45/juice-guard>

Run: `npm install && npm start` → `http://127.0.0.1:3847`

Demo login (README only): `demo@juice.shop / JuiceShop1!`

Figure 5. My hardened login form.

Figure 6. Successful login after validation + CSRF.

OWASP JUICE SHOP

Sign in securely

Demo login with client-side checks and server-side validation — built for a web security assignment.

Email

you@example.com

Email cannot be empty.

Password

At least 8 characters

Password cannot be empty.

Log in

Fix the highlighted fields before submitting.

Figure 7. Empty fields blocked on the client.

OWASP JUICE SHOP

Sign in securely

Demo login with client-side checks and server-side validation — built for a web security assignment.

Email

test@example.com

Password

Password must be at least 8 characters.

Log in

Fix the highlighted fields before submitting.

Figure 8. Short password rejected.

Part 3 — Breaking My Own Form

I tried to break the same form that is in my GitHub repo. First I entered `xss@test.com` and the password `<script>alert(1)</script>`. The client actually allowed the submit (it only checks “@” and length), and the browser sent `POST /api/login`—but the server rejected unsafe characters and no script ran. Next I bypassed the UI with `fetch()` and got the same server error. A login `POST` without an `X-CSRF-Token` returned 403. I also tried `admin@test.com' OR '1'='1`; that was blocked as an invalid email. So classic `XSS/SQLi` did not fully land, but I did expose a real weakness: the client is too trusting on its own.

Learnings (~100 words)

The biggest lesson was that “the attack failed” is not the whole story. My client validation let a script-shaped password through and still called the API. That is exactly how people get hurt when they assume the browser is enough. Calling the endpoint with `fetch()` made the same point louder. `CSRF` testing showed why a random token belongs on state-changing requests. I fixed the gaps by keeping strict server checks, requiring `CSRF` tokens, using `textContent`, and setting a `CSP`. If I build another auth form, I will design the server path first and treat client checks as `UX` only.

Steps and results

XSS attempt: open / → email `xss@test.com` → password `<script>alert(1)</script>` → Log in. *Result:* client submitted the request; server returned an unsafe-character error; no alert.

Client bypass: `fetch('/api/login')` with the same password and a `CSRF` token. *Result:* `HTTP 400` from the server.

CSRF check: `POST` valid credentials with no `X-CSRF-Token`. *Result:* `HTTP 403`.

SQLi probe: email `admin@test.com' OR '1'='1`. *Result:* rejected; no auth bypass.

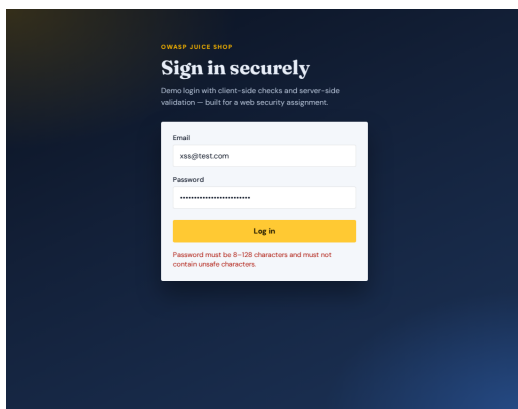


Figure 9. XSS payload passed the client, blocked by the server.

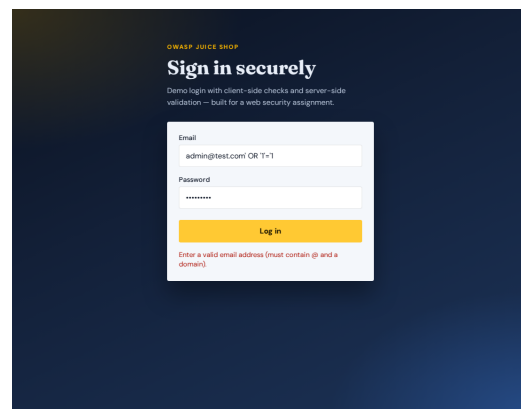


Figure 10. SQL injection–style email rejected.

Weaknesses found in my login form

Tests run against the real `/api/login` endpoint (not a planned page).

1) Missing CSRF token → request rejected

Forged login POST with valid demo credentials but **no** X-CSRF-Token:

```
HTTP 403
{
  "ok": false,
  "error": "Missing or invalid CSRF token."
}
```

Result: CSRF weakness is mitigated — forged request blocked (403).

2) Client-side validation bypass → server still blocks XSS

Called `/api/login` directly with `fetch()`, skipping `validateClient()`, using password `<script>alert(1)</script>`

```
HTTP 400
{
  "ok": false,
  "error": "Password must be 8-128 characters and must not contain unsafe characters."
}
```

Result: client checks are bypassable, but server validation rejected the payload.

Figure 11. API results: CSRF forgery blocked (403); fetch bypass still stopped by server validation.

One fix I applied

Besides server validation, I added CSRF tokens and a Content-Security-Policy so forged requests and inline scripts are harder to abuse even if a UI bug shows up later.